

EVRS Assignment 2 Report

Saharsh Kallu

2024101129

March 2026

1 Overview

This report documents the process of using the VReqDV tool to generate a Unity prototype from JSON specifications for a **DJ Deck** scene. The scene was originally designed manually in Assignment 1, and this assignment recreates it using the model-based specification approach.

The specification files for this assignment are located in `Assets/specifications/version_8/` within the Unity project. This folder contains:

- `article.json` – Object definitions (Table, Turntable.Platter, Vinyl_Record_A, Vinyl_Record_B, Master.Button)
- `behavior.json` – VReqDV behavior specifications with preconditions and actions
- `action-response.json` – Trigger-response pairs for the VReqDV Editor tool
- `UserAlgorithms.cs` – Custom C# logic for snap, play, spin, and removal behaviors

2 Recorded Metrics

2.1 Summary

Metric	Value
Total build time (after requirements finalized)	~90 minutes
Number of development iterations	6
Total requirements mismatched (across all iterations)	9

Table 1: Overall development metrics

2.2 Per-Iteration Breakdown

Iteration	Mismatches	Description
1	3	<code>UserAlgorithms.cs</code> referenced <code>VReqDV.StateAccessor</code> which does not exist before code generation; <code>action-response.json</code> was missing (only <code>behavior.json</code> was created); <code>Vinyl_Record_B</code> used <code>source</code> inheritance which the tool does not support.
2	1	Duplicate code folders (<code>Assets/code/</code> and <code>Assets/Triggers/vreqdv-tool-main/</code>) caused ~ 200 duplicate class compile errors, preventing the VReqDV Editor window from loading.
3	2	<code>NullReferenceException</code> at line 317 due to <code>Vinyl_Record_B</code> having null transform fields (from unsupported <code>source</code> inheritance); objects were oversized relative to camera view.
4	2	<code>Turntable_Platter</code> and <code>Master_Button</code> were too small to be visible in the scene; collision thresholds did not match new object scales.
5	1	<code>Turntable_Platter</code> and <code>Master_Button</code> were positioned inside the Table geometry (Y-position too low relative to table surface).
6	0	All objects correctly positioned above the table. Scene functional.

Table 2: Mismatches per development iteration

3 Challenges Encountered

1. **Compile-time dependency on generated code:** The `VReqDV.StateAccessor` class is only generated after clicking “Display Mock-up”, but `UserAlgorithms.cs` is compiled by Unity immediately. Referencing the non-existent namespace blocked the entire project from compiling, including the VReqDV Editor window itself, creating a chicken-and-egg problem. This was resolved by implementing a standalone state dictionary in `UserAlgorithms.cs`.
2. **JSON schema mismatch:** The VReqDV documentation describes `source` inheritance for articles, but the actual `VReqDV.cs` Editor script does not implement this. Objects with only a `source` field and no transform data caused `NullReferenceException` crashes. Every object had to be fully specified.
3. **VReqDV reads `action-response.json`, not `behavior.json`:** The documentation emphasizes `behavior.json` with preconditions/actions, but the actual Editor tool only reads `action-response.json` with trigger/response pairs. Both files had

to be maintained.

4. **Object sizing and positioning:** Getting the correct scale and Y-position for objects to sit visibly on the table required multiple iterations. Unity’s Cylinder primitive has a default height of 2 units (before scaling), which is non-intuitive when specifying small flat discs.

4 Reflection

4.1 Ease / Difficulty

The specification-based approach is conceptually appealing but presented significant practical friction. Writing JSON by hand for every field (including many nullable/unused fields) was tedious compared to direct scene manipulation in Unity. The tight coupling between JSON schema, C# class definitions, and the Editor tool meant that small mismatches caused cascading failures.

4.2 Clarity of Workflow

The intended workflow (define specs $\rightarrow$ implement logic $\rightarrow$ generate $\rightarrow$ verify $\rightarrow$ iterate) is clear in principle. However, the gap between the *documented* specification format (`behavior.json` with state machines) and the *actual tool implementation* (`action-response.json` with trigger/response pairs) caused confusion. The tool’s reliance on exact string matching for object names, field names, and response types requires careful attention.

4.3 Confidence in Correctness

Moderate. The generated scene matches the specification, but verifying correctness requires running the scene each time. There is no automated validation that the JSON is well-formed with respect to the tool’s expectations before generation.

4.4 Overall Experience

The approach demonstrates the value of separating *what* (specifications) from *how* (implementation), and the versioning system is useful for tracking iterations. However, the current tool maturity means significant manual debugging is still required. For larger scenes with many objects and interactions, the specification approach could save time – but for a small scene like this DJ Deck, direct Unity manipulation in Assignment 1 was faster and more intuitive.

5 Submission Artifacts

- **Unity Package:** Google Drive Link